

AuthentiMark GUI — Technical Study Document

This document serves as a complete study guide and reference for the **AuthentiMark** digital watermarking demo application. It details the core concepts, PyTorch model architectures, backend services, frontend dashboard state, and ethical design principles.

1. Introduction: Digital Watermarking in AI

What is Digital Watermarking?

Digital watermarking is the practice of embedding hidden information (a "watermark") into a carrier signal (such as an image) to verify its authenticity, prove ownership, or trace its origin. In the context of generative AI, active watermarking is used to mark synthetic content at the time of creation to distinguish it from authentic, human-generated media.

Passive Detection vs. Active Watermarking

- **Active Watermarking (AE/VAE)**: Modifies the content during generation by embedding a signature (like a 32-bit binary key). The signature is invisible to the human eye but decodable by a dedicated decoder neural network.
- **Passive Detection (ViT)**: Analyzes an image after the fact to classify whether it contains a watermark, without needing to extract the original message key.

2. Reconstructed Model Architectures

The PyTorch checkpoints saved from training contain weights mapped to specific neural network architectures. Because default imports from standard libraries expect different parameter naming conventions, the architectures were reconstructed from scratch in [backend/models.py](#).

A. Residual AutoEncoder (AE)

The AutoEncoder consists of two parts: an **Encoder** and a **Decoder**.

1. AEEncoder

- **What it does**: Embeds a 32-bit binary message (e.g. `[1, 0, 1, 1, ...]`) into a $128 \times 128 \times 3$ image.
- **How it works**:

1. The 32-bit message is passed through a Linear layer (`msg_fc`) and reshaped to $128 \times 128 \times 1$.

2. This message feature map is concatenated with the 3-channel input image along the channel axis, creating a $128 \times 128 \times 4$ tensor.

3. The concatenated tensor is passed through a convolutional network (**net**) consisting of Conv2D, BatchNorm2D, and ReLU activations, outputting a 3-channel residual map.

4. The watermarked image is produced as:

$$\text{Watermarked} = \text{Image} + (\text{residual_scale} \times \text{residual})$$

- **Why we use `residual_scale=0.3`:** The residual scale controls the trade-off between **imperceptibility** and **robustness**. A smaller scale (like 0.3) ensures the watermark remains invisible to human observers, while still providing enough signal for the decoder to extract.

2. AEDecoder

- **What it does:** Reconstructs the 32-bit key from a watermarked image.
- **How it works:**
 - Employs convolutional layers with a stride of 2 to downsample the image scale ($128 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 8$).
 - Flattens the final $8 \times 8 \times 128$ feature tensor to 8192 dimensions.
 - Passes the features through linear layers (**fc**) to output 32 raw logit values. Logits ≥ 0 decode to **1**, and logits < 0 decode to **0**.

B. Variational AutoEncoder (VAE)

The VAE shares a similar purpose with the standard AE but models the embedding space as a continuous probability distribution.

1. VAEEncoder

- **How it differs from AE:** Instead of directly mapping the input to a deterministic residual map, the VAE encoder passes features through a convolutional trunk to produce two tensors:
 - **Mean (μ)** of the distribution.
 - **Log-Variance ($\log \sigma^2$)** of the distribution.
- **Reparameterization Trick:** To allow backpropagation during training through a random sampling process, the VAE samples latent variables (z) as:

$$z = \mu + \epsilon \times \sigma \quad \text{where} \quad \epsilon \sim \mathcal{N}(0, I)$$

- **Why we use it:** Modeling the message embeddings as standard normal distributions improves generalization, smoothing the latent spaces so that watermarked images look more natural.

2. VAEDecoder

- Matches the **AEDecoder** architecture, operating as a downsampling classifier to extract the 32 logits representing the watermark bits.

C. Vision Transformer (ViT) Detector

The **WatermarkDetector** is a binary classifier trained to check if an image is watermarked.

1. ViT Backbone

- Loaded using fine-tuned weights based on the standard [google/vit-base-patch16-224](#) configuration.
- Resizes the input image to 224 \times 224 \times 3.
- Divides the image into patches of size 16 \times 16 (total 196 patches).
- Projects each patch into a 768-dimensional space.
- Prepends a learnable classification token (**CLS token**) and adds position embeddings.
- Passes the sequence through 12 self-attention transformer blocks.

2. CLS Token Classification Head

- **Why we use the CLS token, not the pooler:** The CLS token accumulates global contextual information across all self-attention layers. In fine-tuning tasks, standard practice extracts the CLS token's final hidden state (1 \times 768) and passes it directly to a custom classification head (`nn.Sequential(nn.Linear(768, 128), nn.ReLU(), nn.Linear(128, 2))`), bypassing average pooling which can wash out high-frequency watermark signatures.

3. Backend Architecture: FastAPI

The Python backend ([backend/main.py](#)) serves model inference behind a REST API.

Key Performance Patterns

- **Startup Loading:** All heavy PyTorch state dictionaries are loaded into RAM once at startup ([main.py](#) and [inference.py](#)). Requests perform inference only, making the application responsive.
- **CORS Configuration:** Restricts access specifically to the Vite frontend origin (<http://localhost:5173>) to prevent unauthenticated cross-origin resource requests.

Endpoint Breakdown

1. **GET /health:** Returns the loading status of the three models, ensuring checkpoints are present.
2. **POST /watermark:**
 - Takes a cover image and the method selection ("**ae**" or "**vae**").
 - Generates a random 32-bit key.
 - Encodes and saves the watermarked image as a base64 PNG data URL.
3. **POST /simulate-attack:**
 - Takes the watermarked image, an attack type, and intensity.
 - Applies PIL-based image distortions:

- **Crop**: Crops the center to a ratio (e.g. 80%) and rescales.
- **Rotate**: Rotates by degree.
- **Gaussian Noise**: Adds standard normal deviations using Numpy.
- **Gaussian Blur**: Filters via Pillow GaussianBlur.
- **JPEG Compression**: Saves as JPEG with a lossy quality factor.
- Returns the distorted image as a base64 PNG data URL.

4. **POST /detect**:

- Preprocesses the image to 224x224, runs detector inference, and applies softmax.
- Returns the prediction index, confidence percentage, and ethics-aware verdict.

4. Frontend Architecture: React, Vite, and Tailwind

The frontend ([frontend/](#)) is built as a single-page React app styled with Tailwind CSS.

Shared State & Tab Transitions

- **Lifting State**: To allow a seamless E2E user verification flow, state variables like **detectFile** are lifted to the root **App.jsx** component.
- **Workflow**:
 1. A user watermarks an image.
 2. Under section 3 (Distortion), the user clicks **"Apply Attack"**.
 3. The user click **"Send to Detector for Testing"**.
 4. The handler converts the base64 attacked image to a file object, updates the lifted **detectFile** state, and switches the active tab index to **detect**.
 5. The Detector panel receives the file and clears previous verdict states, preparing a clean slate for the verification run.

Premium Dark Theme

We implemented a dark, luxury UI theme modeled after premium media verification applications:

- **#0e0c0a (Background)**: Solid dark charcoal color.
- **#161412 (Card Panels)**: Elevated dark grey/brown surfaces.
- **#c5a880 (Accent Color)**: Warm gold-beige highlight for buttons, selection dropdown outlines, slider progress bars, active tabs, and focus outlines.
- **#f7ebd8 (Typography)**: Ivory/cream serif typeface (**font-serif**) for headers.

5. Ethical Considerations: Verdict Safeguards

In digital forensics, presenting binary ("Yes" or "No") classification outputs can lead to false accusations of plagiarism or content forging.

Softmax Probability & Confidence

Softmax outputs represent the model's confidence distribution over the classes (0: Unwatermarked, 1: Watermarked). If the model predicts class 1 with 51% confidence, it is mathematically the highest class, but practically highly uncertain.

The 75% Safeguard Threshold

In `backend/inference.py`, the `verdict_from_confidence` helper maps scores to safe assertions:

- **Confidence $\geq$ 75%:** Returns **"Likely watermarked"** or **"Likely not watermarked"**.
- **Confidence < 75%:** Returns **"Uncertain -- do not treat as definitive proof"**.

Surfacing this warning ensures that users are notified when the detector model is experiencing high entropy (uncertainty), forcing them to verify content through other channels rather than relying on low-confidence outputs.